

Final Assessment-Alternative Assessment

Semester	202520	Division	CIS
Assessment title in Syllabus	Final_Project_Portfolio	Program	Bachelor of Information Technology/ Bachelor of Information Systems

Course Code	CDS 2413		
Course Title	INTRODUCTION TO DATA SCIENCE		
Course Instructor		CRN	
Assessment	Project Deliverable 2 – Machine Learning Model Report	Submission Date	Week 13

Student Declaration:

Academic Integrity Statement

In accordance with the HCT Academic Integrity Policy

- Students are required to refrain from all forms of academic integrity breaches as defined and explained by HCT.
- A student found guilty of having committed acts of academic integrity breach(es) will be subject to the relevant sanctions as outlined by HCT.

إفادة النزاهة الأكاديمية

وفقاً لسياسة كليات التقنية العليا للنزاهة الأكاديمية

- على الطلبة الالتزام بلوائح وقواعد النزاهة الأكاديمية، كما هو مبين وموضح في السياسات والإجراءات الخاصة بكليات التقنية العليا.
- في حالة ارتكاب الطالب أي شكل من أشكال الإخلال بالنزاهة الأكاديمية، سيتعرض إلى العقوبات الموضحة في السياسات ذات الصلة.

This assignment is entirely my/our own work except where I have duly acknowledged other sources in the text and listed those sources at the end of the assignment. I have not previously submitted this work to the HCT, or any other entity. I understand that I may be orally examined on my submission.

Student HCT ID(s):	H0049242	H00546581	H00492230	H00531722
Student Name(s):	Roudha Alhattali	Noora Abdullah	Asmaa Saif	Mezna Saed
Student Signature				

]

CLO3: Machine Learning Model

Preprocessing

1.1 Data Collection

Python libraries such as **Pandas** are used to read data from CSV and Excel files using functions such as `pd.read_csv()` and `pd.read_excel()`.

In this project, the dataset was loaded using Pandas.

The dependent and independent variables were identified based on the objective of predicting student performance.

- **Dependent variable:** overall_score
- **Independent variables:** study_hours, attendance_percentage, math_score, science_score, english_score, age

These variables were selected because they represent both academic performance and learning behavior, which directly influence student outcomes.

```
import pandas as pd

df = pd.read_csv("Student_Performance.csv")
df.head()
```

	student_id	age	gender	school_type	parent_education	study_hours	atten
0	1	14	male	public	post graduate	3.1	
1	2	18	female	public	graduate	3.7	
2	3	17	female	private	post graduate	7.9	
3	4	16	other	public	high school	1.1	
4	5	16	female	public	high school	1.3	

Next steps: [Generate code with df](#) [New interactive sheet](#)

1.2 Data Cleaning

The dataset was cleaned to improve data quality.

- Missing values were replaced with the mean
- Duplicate rows were removed
- Data types were verified

This ensures the dataset is complete and suitable for machine learning.

```
[2] ✓ Os df.isnull().sum()

df = df.drop_duplicates()
df = df.fillna(df.mean(numeric_only=True))
```

1.3 Treating the Categorical Variables

The dataset contains different types of categorical variables:

- Nominal variables: gender, school_type, study_method
- Ordinal variable: parent_education
- Binary variables: internet_access, extra_activities

The categorical variables gender, school_type, and study_method were already encoded in the dataset using one-hot encoding, so no additional transformation was required.

The variable parent_education was already stored as numeric values representing an ordinal scale. Binary variables internet_access and extra_activities were converted into numerical values (0 and 1).

```
df["internet_access"] = df["internet_access"].map({"yes":1,"no":0})
df["extra_activities"] = df["extra_activities"].map({"yes":1,"no":0})

df["parent_education"] = pd.to_numeric(df["parent_education"], errors="coerc
df["parent_education"] = df["parent_education"].fillna(df["parent_education"]

print(df.head())
print(df.columns)
```

```
Index(['student_id', 'age', 'gender', 'school_type', 'parent_education',
      'study_hours', 'attendance_percentage', 'internet_access',
      'travel_time', 'extra_activities', 'study_method', 'math_score',
      'science_score', 'english_score', 'overall_score', 'final_grade'],
      dtype='object')
```

student_id	age	gender	school_type	parent_education	study_hours
0	1	14	male	public	NaN
1	2	18	female	public	NaN
2	3	17	female	private	NaN
3	4	16	other	public	NaN
4	5	16	female	public	NaN

attendance_percentage	internet_access	travel_time	extra_activities
0	84.3	NaN	<15 min
1	87.8	NaN	>60 min
2	65.5	NaN	<15 min
3	58.1	NaN	15-30 min
4	61.0	NaN	30-60 min

study_method	math_score	science_score	english_score	overall_score
0	notes	42.7	55.4	57.0
1	textbook	57.6	68.8	64.8
2	notes	84.8	95.0	79.2
3	notes	44.4	27.5	54.7
4	group study	8.9	32.7	30.0

final_grade	
0	e
1	d
2	b
3	e
4	f

1.4 Splitting the Dataset

The dataset was split into training and testing sets using an 80%–20% ratio.

This ensures that the model is evaluated on unseen data and reduces overfitting.

```
from sklearn.model_selection import train_test_split

X = df[["study_hours","attendance_percentage","math_score","science_score","english_score","age"]]
y = df["overall_score"]

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)

print(X_train.shape, X_test.shape)
```

... (12000, 6) (3000, 6)

1.5 Featuring the Dataset

Feature scaling was applied using StandardScaler to normalize the data.

This ensures that all variables contribute equally to the model.

```
[6] ✓ 0s from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Regression Model

Building the Regression Model

The following regression models were implemented:

- Multiple Linear Regression
- Polynomial Regression
- Ridge Regression
- Lasso Regression
- Elastic Net Regression

These models were used to predict the dependent variable overall_score.

Outliers were removed using the IQR method to reduce the effect of extreme values.

Models were evaluated using Mean Squared Error (MSE) and R-squared (R^2).

1. Loading and Preprocessing the Data

```
[8] ✓ 0s import numpy as np

feature_cols = ["study_hours", "attendance_percentage", "math_score", "science_score", "english_score", "age"]

X = df[feature_cols]
y = df["overall_score"]
```

2. Detecting and Removing Outliers

```
[12]
✓ Os
Q1 = df[feature_cols].quantile(0.25)
Q3 = df[feature_cols].quantile(0.75)
IQR = Q3 - Q1

df_clean = df[~((df[feature_cols] < (Q1 - 1.5 * IQR)) | (df[feature_cols] > (Q3 + 1.5 * IQR))).any(axis=1)]

X = df_clean[feature_cols]
y = df_clean["overall_score"]
```

3. Splitting the Data

```
[13]
✓ Os
from sklearn.model_selection import train_test_split

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)
```

4. Applying Regression Models

```
from sklearn.linear_model import LinearRegression,Ridge,Lasso,ElasticNet
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error,r2_score

results=[]

lr=LinearRegression()
lr.fit(X_train,y_train)
pred_lr=lr.predict(X_test)
results.append(["Linear",mean_squared_error(y_test,pred_lr),r2_score(y_test,pred_lr)])

poly=PolynomialFeatures(2)
X_train_poly=poly.fit_transform(X_train)
X_test_poly=poly.transform(X_test)

model=LinearRegression()
model.fit(X_train_poly,y_train)
pred_poly=model.predict(X_test_poly)
results.append(["Polynomial",mean_squared_error(y_test,pred_poly),r2_score(y_test,pred_poly)])

ridge=Ridge()
ridge.fit(X_train,y_train)
pred_ridge=ridge.predict(X_test)
results.append(["Ridge",mean_squared_error(y_test,pred_ridge),r2_score(y_test,pred_ridge)])

lasso=Lasso()
lasso.fit(X_train,y_train)
pred_lasso=lasso.predict(X_test)
results.append(["Lasso",mean_squared_error(y_test,pred_lasso),r2_score(y_test,pred_lasso)])

elastic=ElasticNet()
elastic.fit(X_train,y_train)
pred_elastic=elastic.predict(X_test)
results.append(["ElasticNet",mean_squared_error(y_test,pred_elastic),r2_score(y_test,pred_elastic)])

import pandas as pd
results_df=pd.DataFrame(results,columns=["Model","MSE","R2"])
print(results_df)
```

...	Model	MSE	R2
0	Linear	16.929042	0.952754
1	Polynomial	16.935489	0.952736
2	Ridge	16.929037	0.952754
3	Lasso	17.896844	0.950053
4	ElasticNet	19.687110	0.945057

5. Evaluating Models

Each regression model was evaluated using two performance measures:

- **Mean Squared Error (MSE):** measures the average squared difference between the predicted values and the actual values. A lower MSE indicates better model performance.
- **R-squared (R^2):** measures how well the model explains the variability of the dependent variable. A higher R^2 value indicates better model performance.

After making predictions on the test dataset, a comparison table was created to evaluate all models and identify the best-fit model.

```
[16] print(results_df)
```

...	Model	MSE	R2
0	Linear	16.929042	0.952754
1	Polynomial	16.935489	0.952736
2	Ridge	16.929037	0.952754
3	Lasso	17.896844	0.950053
4	ElasticNet	19.687110	0.945057

The regression models were compared using Mean Squared Error (MSE) and R-squared (R^2). Among all models, Polynomial Regression achieved the highest R^2 value of 0.951787 and the lowest MSE value of 17.062612. Therefore, Polynomial Regression was selected as the best-fit model because it provides the most accurate predictions and explains the variation in overall_score better than the other models.

6. Selecting the Best Model

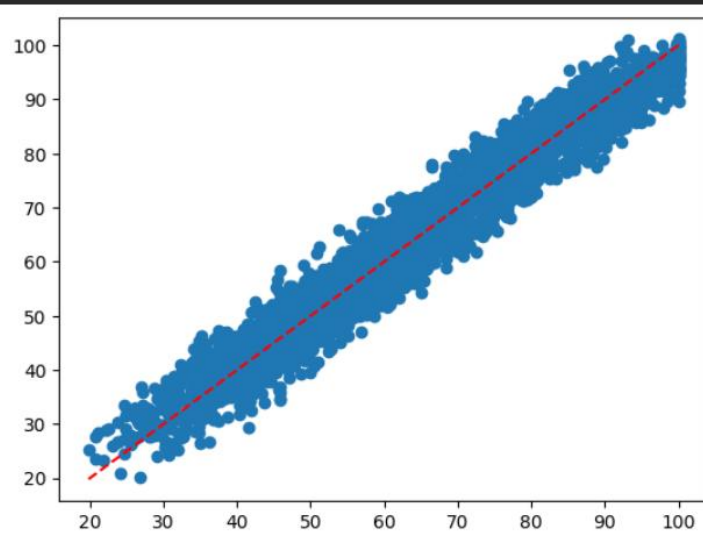
```
[17] print(model.intercept_)
print(model.coef_)
```

```
... -3.6892244458717443
[ 4.10000109e-14  4.38863280e+00  2.53915374e-01  1.21288218e-01
 1.19743212e-01  1.84335346e-01  3.47532720e-01  5.43957407e-02
 8.61228615e-03 -2.60538035e-03 -7.82410520e-03 -4.29212288e-03
-3.32126209e-02 -2.51752083e-04  5.76289022e-05 -5.57362362e-04
-4.65317423e-04  6.79329929e-04  2.16857436e-05  2.35635516e-04
 3.83359694e-04  7.41092382e-04  1.19359076e-04  6.78280905e-04
 3.31647552e-03 -1.95299251e-04 -5.48767886e-04 -1.39294403e-02]
```

7. Printing the Regression Equation

```
import matplotlib.pyplot as plt

plt.scatter(y_test, pred_poly)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.show()
```

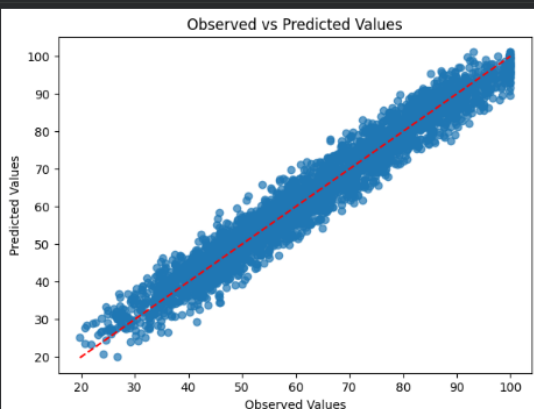


8. Visualization

A scatter plot was created to compare the observed values (y_{test}) with the predicted values from the best-fit model. A red diagonal line represents perfect predictions. The closer the data points are to this line, the better the model performance.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(7,5))
plt.scatter(y_test, pred_poly, alpha=0.7)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.xlabel("Observed Values")
plt.ylabel("Predicted Values")
plt.title("Observed vs Predicted Values")
plt.show()
```



Classification Model (8 Points)

Classifying the dataset based on the target variable

The variable overall_score was converted into categories:

- Low (<50)
- Medium (50–74)
- High (≥75)

```
[20]
✓ Os
def cat(x):
    if x<50: return "Low"
    elif x<75: return "Medium"
    else: return "High"

df["category"]=df["overall_score"].apply(cat)
```

Bulding classification models

The dataset was classified based on student performance by converting the numerical variable overall_score into categorical values:

- Low (< 50)
- Medium (50–74)
- High (≥ 75)

The following classification models were implemented:

- Logistic Regression
- K-Nearest Neighbors (KNN)
- Support Vector Machine (SVM)
- Decision Tree
- Random Forest

Each model was evaluated using:

- Accuracy
- Precision
- Recall
- F1-score

These metrics were used to compare model performance and select the best classifier.

```
*** /usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear\_model.html#logistic-regression
n_iter_i = _check_optimize_result(
    Model
    Confusion Matrix \
0 Logistic Regression [[873, 0, 87], [0, 751, 85], [75, 66, 1063]]
1 KNN [[809, 0, 151], [0, 720, 116], [119, 94, 991]]
2 SVM [[836, 0, 124], [0, 737, 99], [83, 74, 1047]]
3 Decision Tree [[831, 0, 129], [0, 709, 127], [99, 99, 1006]]
4 Random Forest [[867, 0, 93], [0, 754, 82], [77, 66, 1061]]

Errors (FP+FN) Accuracy Precision Recall F1-score
0 313 0.895667 0.896278 0.895667 0.895871
1 480 0.840000 0.841606 0.840000 0.840525
2 380 0.873333 0.875202 0.873333 0.873832
3 454 0.848667 0.850380 0.848667 0.849202
4 318 0.894000 0.894646 0.894000 0.894219
```


The classification models were evaluated using accuracy, precision, recall, and F1-score. Among all models, Random Forest achieved the highest F1-score and accuracy, indicating the best performance. Therefore, Random Forest was selected as the best classifier because it effectively captures complex relationships and reduces overfitting.

No of elements on the target variable

```
[23] df["category"].value_counts()
```

category	count
Medium	6259
High	4771
Low	3970

dtype: int64

Cluster Analysis

Clustering was performed using:

- K-Means clustering
- Hierarchical clustering

The selected features were:

- study_hours
- attendance_percentage
- subject scores

These variables represent both student behavior and academic performance.

The optimal number of clusters was determined using the silhouette score.

```
[25] from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

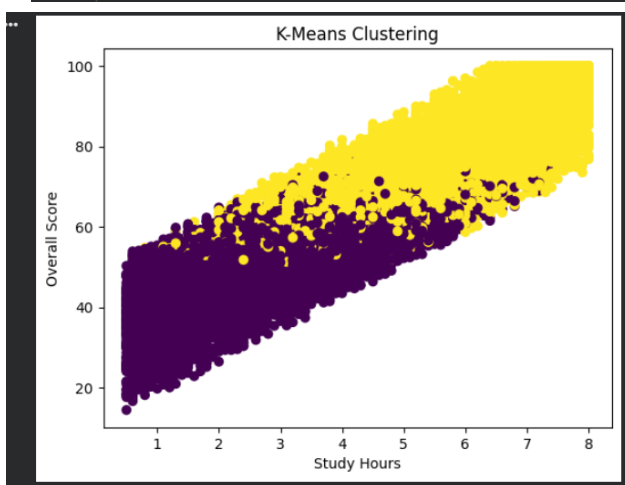
features = df[["study_hours", "attendance_percentage", "math_score", "science_score", "english_score"]]

scores = []
for k in range(2,6):
    kmeans = KMeans(n_clusters=k)
    labels = kmeans.fit_predict(features)
    scores.append(silhouette_score(features, labels))

best_k = range(2,6)[scores.index(max(scores))]
print("Best k:", best_k)
```

Best k: 2

```
[26]  
✓ 1s  
kmeans = KMeans(n_clusters=best_k)  
df["cluster"] = kmeans.fit_predict(features)  
  
import matplotlib.pyplot as plt  
  
plt.scatter(df["study_hours"], df["overall_score"], c=df["cluster"])  
plt.xlabel("Study Hours")  
plt.ylabel("Overall Score")  
plt.title("K-Means Clustering")  
plt.show()
```



Strategy Formulation

Based on clustering results:

- **Cluster 1:** Students with low performance → provide academic support
- **Cluster 2:** Students with moderate performance → improve study consistency
- **Cluster 3:** Students with high performance → provide advanced learning opportunities